23CSE473

Foundations of Deep Learning Laboratory Exercises

HARI SREERAM R
CH.SC.U4CSE23019

These exercises cover the fundamental concepts of deep learning, including perceptron, multi-layer perceptron, loss functions, backpropagation, optimization, and regularization.

Exercise 1: Evolution from Perceptron to Multi-Layer Perceptron (MLP)

Objective
Understand why the single-layer perceptron failed for certain problems and how the Multi-Layer Perceptron (MLP) overcame these limitations.

Tasks
1. Explain the architecture of a single layer perceptron.

Input layer

- One output neuron
- No hidden layer

Each input is multiplied by a weight, summed together with a bias, and passed through an activation function. Mathematically,

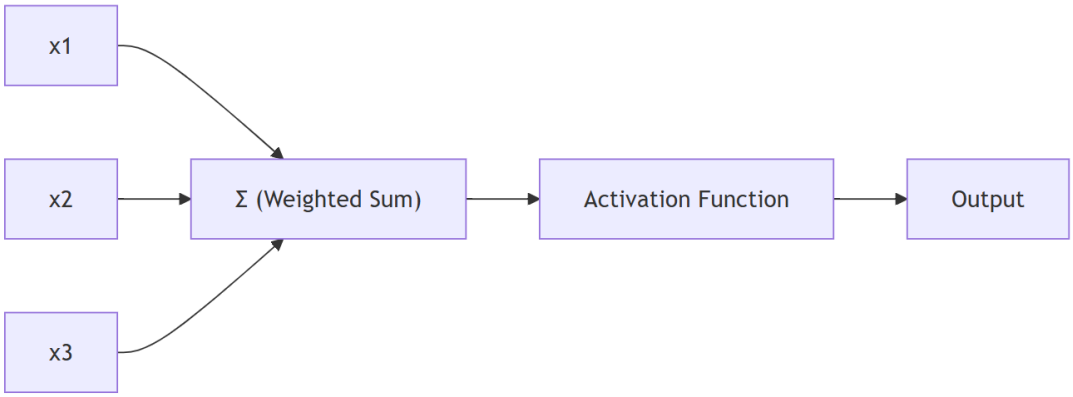
$$z = \sum_{i=1}^{n} w_i x_i + b$$

this is the equation of a single layer perceptron. The z is then passed to an activation function to get the final output. And thus, the output becomes,

$$y = f(z)$$

- w_i = weights
- x_i = inputs
- b = bias
- f(z) = activation function

The architecture of a single-layer perceptron with three input neurons is shown below. Although the figure illustrates three inputs, the perceptron can be generalized to handle any number of input features by adding corresponding weighted connections to the summation unit.



The perceptron creates only one linear decision boundary.

2. Draw the XOR truth table.

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

3. And Gate

The AND gate is a basic logic gate that produces an output of 1 only when both inputs are 1. If either or both inputs are 0, the output is 0. In the plotted graph, the three points (0,0), (0,1), and (1,0) belong to Class 0, while only (1,1) belongs to Class 1. The decision boundary clearly separates the single positive class from the negative class, demonstrating that the AND gate is linearly separable. A single-layer perceptron can correctly classify all input combinations using a straight-line decision boundary.

Truth Table:

Input (x_1)	**Input (x_2)**	**Output**
0	0	0
0	1	0
1	0	0
1	1	1

CODE:

and.py > ...

```python
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------
# AND Gate Dataset
# ------------------------------
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 0, 0, 1])

# Perceptron weights for AND
w = np.array([1, 1])
b = -1.5

# ------------------------------
# Decision Boundary
# ------------------------------
xx = np.linspace(-0.2, 1.2, 200)
yy = -(w[0] * xx + b) / w[1]

plt.figure(figsize=(6,6))

# Plot Class 0
plt.scatter(
    X[y==0,0],
    X[y==0,1],
    s=120,
    marker='o',
    label='Class 0'
)

```

```
# Plot Class 1
plt.scatter(
    X[y==1,0],
    X[y==1,1],
    s=150,
    marker='^',
    label='Class 1'
)

# Decision Boundary
plt.plot(xx, yy, linewidth=2, label='Decision Boundary')

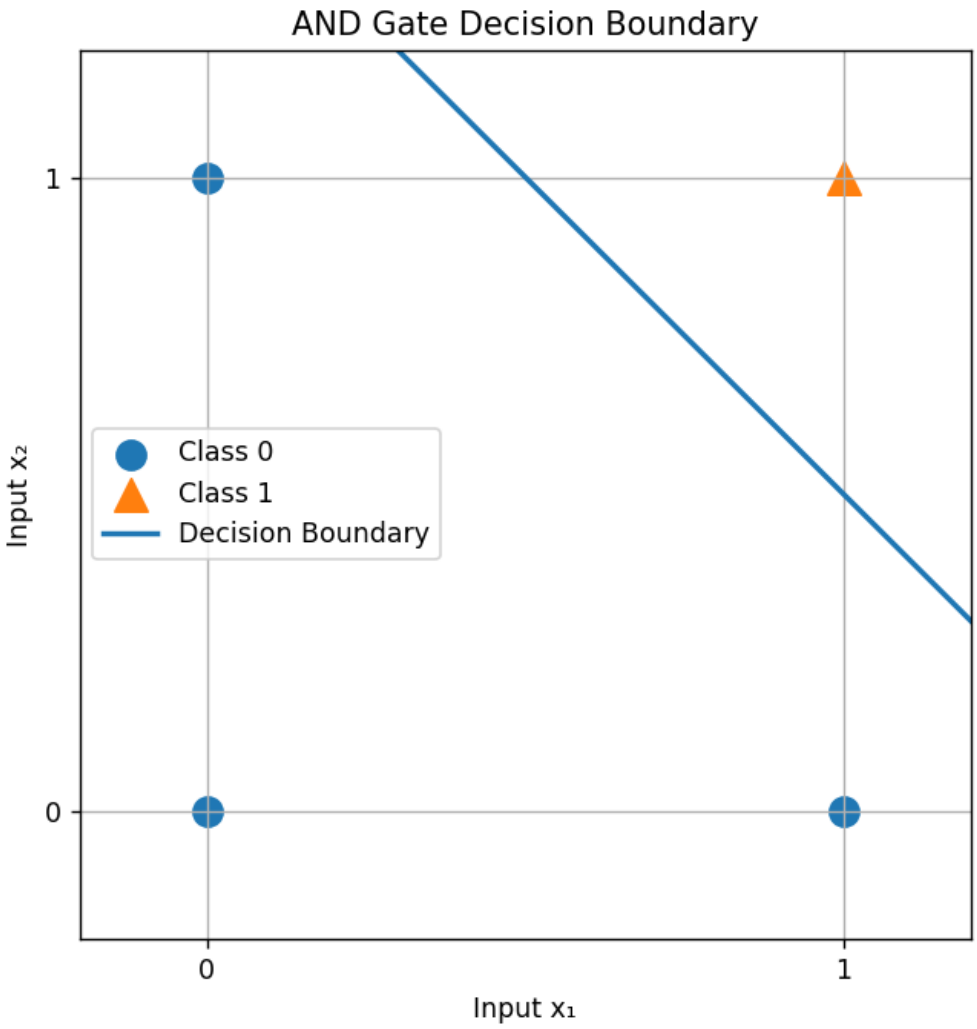
plt.title("AND Gate Decision Boundary")
plt.xlabel("Input x₁")
plt.ylabel("Input x₂")

plt.xlim(-0.2,1.2)
plt.ylim(-0.2,1.2)

plt.xticks([0,1])
plt.yticks([0,1])

plt.grid(True)
plt.legend()

plt.savefig("AND_Decision_Boundary.png", dpi=300, bbox_inches="tight")
plt.show()
```


AND Gate Decision Boundary
Class 0
Class 1
Decision Boundary
Input x₂
Input x₁
0
1
0
1

4. Or Gate

The OR gate is a logic gate that produces an output of 1 whenever at least one input is 1. The output is 0 only when both inputs are 0. In the graph, the point (0,0) belongs to Class 0, while (0,1), (1,0), and (1,1) belong to Class 1. The decision boundary separates the single negative class from the positive class, showing that the OR gate is also linearly separable. Like the AND gate, it can be implemented using a single-layer perceptron with an appropriate linear decision boundary.

Truth table:

Input (x_1)	**Input (x_2)**	**Output**
0	0	0
0	1	1
1	0	1
1	1	1

CODE:

or.py > ...

```python
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# OR Gate Dataset
# ----------------------------
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,1])

# Perceptron weights for OR
w = np.array([1,1])
b = -0.5

# ----------------------------
# Decision Boundary
# ----------------------------
xx = np.linspace(-0.2,1.2,200)
yy = -(w[0]*xx + b)/w[1]

plt.figure(figsize=(6,6))

# Plot Class 0
plt.scatter(
    X[y==0,0],
    X[y==0,1],
    s=120,
    marker='o',
    label='Class 0'
)

```

```python
# Plot Class 1
plt.scatter(
    X[y==1,0],
    X[y==1,1],
    s=150,
    marker='^',
    label='Class 1'
)

# Decision Boundary
plt.plot(xx, yy, linewidth=2, label='Decision Boundary')

plt.title("OR Gate Decision Boundary")
plt.xlabel("Input $x_1$")
plt.ylabel("Input $x_2$")

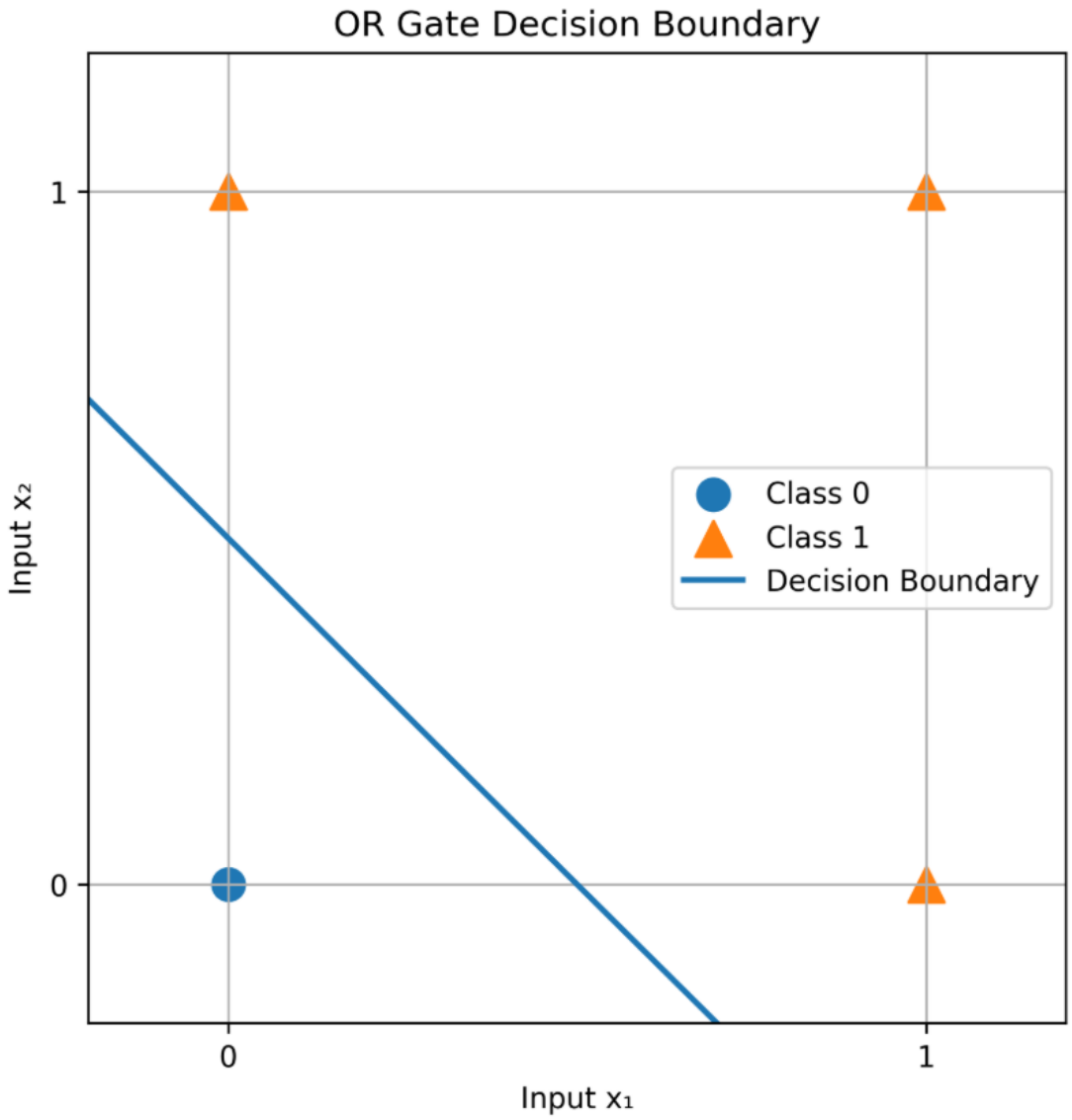
plt.xlim(-0.2,1.2)
plt.ylim(-0.2,1.2)

plt.xticks([0,1])
plt.yticks([0,1])

plt.grid(True)
plt.legend()

plt.savefig("OR_Decision_Boundary.png", dpi=300, bbox_inches="tight")
plt.show()
```

OUTPUT:


OR Gate Decision Boundary
1
0
Input x₂
0
1
Input x₁
Class 0
Class 1
Decision Boundary

5. Plot the XOR data points in a 2D coordinate system.
CODE:

xor_plot.py 1

C: > Users > HARI RAMESH > Desktop > DL > DLLab > XOR > xor_plot.py > ...

```python
import matplotlib.pyplot as plt

# XOR dataset
class0_x = [0, 1]
class0_y = [0, 1]

class1_x = [0, 1]
class1_y = [1, 0]

# Create figure
plt.figure(figsize=(6, 6))

# Plot Class 0 (Negative)
plt.scatter(class0_x, class0_y, color='red', marker='x',
            s=120, label='Class 0')

# Plot Class 1 (Positive)
plt.scatter(class1_x, class1_y, color='blue', marker='o',
            s=120, label='Class 1')

# Label each point
points = [(0,0), (0,1), (1,0), (1,1)]
for x, y in points:
    plt.text(x + 0.03, y + 0.03, f"({x}, {y})", fontsize=10)

# Cartesian axes through the origin
plt.axhline(0, color='black', linewidth=1.5)
plt.axvline(0, color='black', linewidth=1.5)

# Axis limits and ticks
plt.xlim(-0.2, 1.2)
plt.ylim(-0.2, 1.2)
plt.xticks([0, 1])
plt.yticks([0, 1])

# Labels and title
plt.xlabel("Input Feature $x_1$", fontsize=12)
plt.ylabel("Input Feature $x_2$", fontsize=12)
plt.title("Figure 1: XOR Dataset", fontsize=14, fontweight="bold")

# Remove grid
plt.grid(False)
```

```
# Labels and title
plt.xlabel("Input Feature $x_1$", fontsize=12)
plt.ylabel("Input Feature $x_2$", fontsize=12)
plt.title("Figure 1: XOR Dataset", fontsize=14, fontweight="bold")

# Remove grid
plt.grid(False)

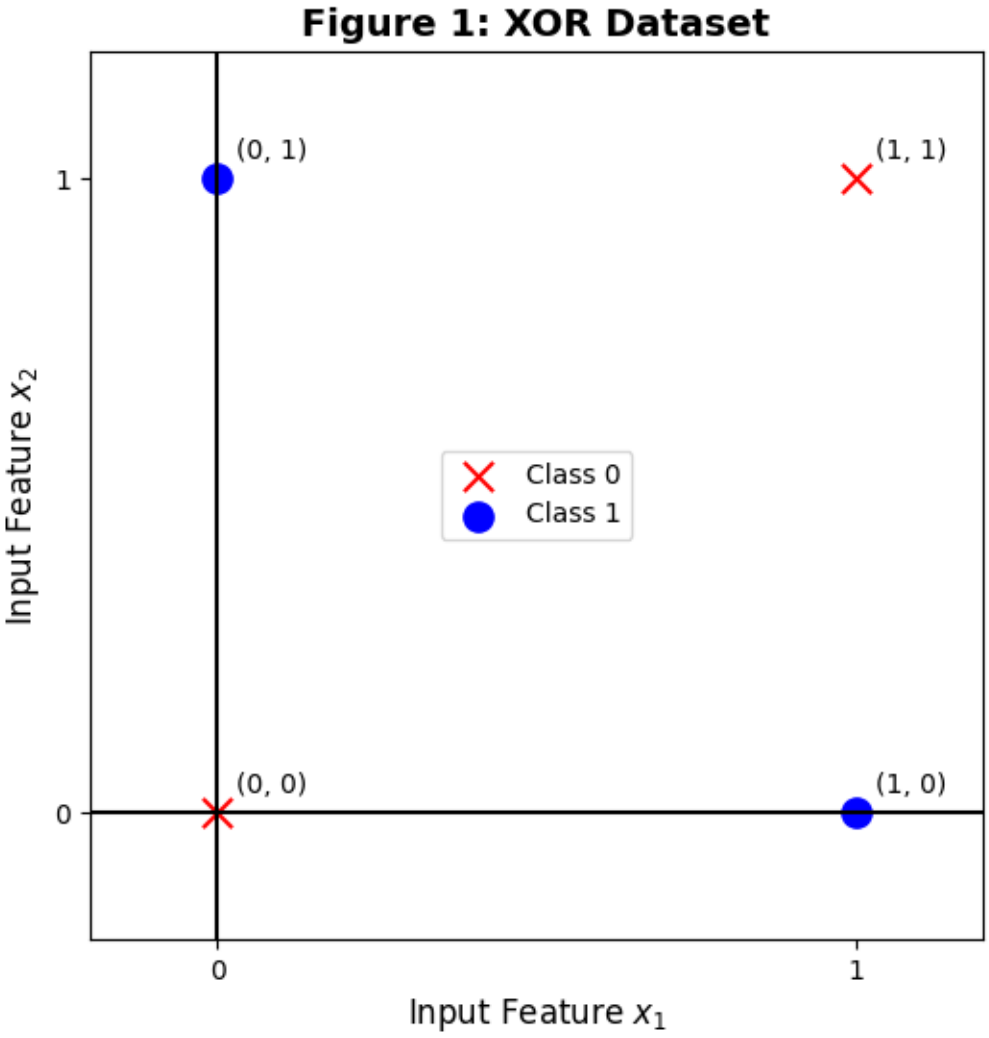
# Legend
plt.legend(loc="center")

# Save the figure
plt.savefig(
    "Figure_1_XOR_Dataset.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```

OUTPUT:



6. Prove mathematically that XOR is not linearly separable.

   A **single-layer perceptron** can correctly classify a dataset only if the data is **linearly separable**, meaning that a single straight line can separate the two classes.

For the XOR problem, the dataset is:

Input (x_1, x_2)	Output
(0,0)	0
(0,1)	1
(1,0)	1
(1,1)	0

Assume that there exists a linear classifier capable of perfectly classifying the XOR dataset.

The equation of a linear decision boundary is

$$w_1 x_1 + w_2 x_2 + b = 0$$

where,

- w_1 and w_2 are the weights,
- b is the bias.

For correct classification, the classifier must satisfy the following conditions:

- Output **0**:

  $$w_1 x_1 + w_2 x_2 + b < 0$$

- Output **1**:

  $$w_1 x_1 + w_2 x_2 + b > 0$$

Step 1: Apply the Conditions to Each XOR Input

Case 1: Point (0,0)

The desired output is **0**.

Substituting the coordinates,

$$w_1(0) + w_2(0) + b < 0$$

Therefore,

$$b < 0$$

Case 2: Point (1,0)

The desired output is **1**.

Substituting,

$$w_1(1) + w_2(0) + b > 0$$

which simplifies to

$$w_1 + b > 0$$

Case 3: Point (0,1)

The desired output is **1**.

Substituting,

$$w_1(0) + w_2(1) + b > 0$$

Therefore,

$$w_2 + b > 0$$

Case 4: Point (1,1)

The desired output is **0**.
Substituting,

$$w_1 + w_2 + b < 0$$

Thus,

$$w_1 + w_2 + b < 0$$

Step 2: Combine the Inequalities
From Cases 2 and 3,

$$w_1 + b > 0$$

and

$$w_2 + b > 0$$

Adding these two inequalities gives

$$(w_1 + b) + (w2 + b) > 0$$

or

$$w_1 + w_2 + 2b > 0$$

Step 3: Use the Result from Case 1
From Case 1,

$$b < 0$$

which means

$$- b > 0$$

Now

$$(w_1 + w_2 + 2b) - b = w_1 + w_2 + b$$

Since

$$w_1 + w_2 + 2b > 0$$

and

$$- b > 0,$$

adding the positive quantity (−b) gives

$$w_1 + w_2 + b > 0$$

Thus,

$$w_1 + w_2 + b > 0$$

Step 4: Contradiction
However, from Case 4 we already obtained

$$w_1 + w_2 + b < 0$$

Therefore, we have reached two contradictory statements:

$$w_1 + w_2 + b > 0$$

and

$$w_1 + w_2 + b < 0$$

Both cannot be true simultaneously.
Hence, our original assumption that a single linear classifier exists must be false.

Since no values of the weights w_1 , w_2 , and bias b satisfy all four inequalities simultaneously, no single linear decision boundary can correctly classify all XOR input combinations.

Therefore, The XOR function is not linearly separable.

This mathematical contradiction proves that a single-layer perceptron cannot solve the XOR problem. To correctly classify XOR, a neural network requires at least one hidden layer with a nonlinear activation function, such as a Multi-Layer Perceptron (MLP), which can learn nonlinear decision boundaries.

7. Explain why no single linear decision boundary can correctly classify XOR.

A single-layer perceptron classifies data by drawing one linear decision boundary, which is represented by a straight line in a two-dimensional feature space. For a dataset to be correctly classified by such a model, all data points belonging to the same class must lie on the same side of the decision boundary.

In the XOR dataset, the two classes are arranged diagonally:

- Class 1 (Positive): (0,1) and (1,0)
- Class 0 (Negative): (0,0) and (1,1)

This arrangement creates a crisscross pattern where points of different classes alternate across the feature space. Because of this diagonal distribution, any straight line drawn through the plane will inevitably place at least one point from each class on the wrong side of the boundary.

For example, a vertical or horizontal line can separate either the left and right points or the top and bottom points, but it cannot simultaneously isolate the two diagonal points belonging to the same class. Similarly, a diagonal line will correctly classify one pair of points while misclassifying the other pair.

Since a straight line can divide the plane into only two half-spaces, it is impossible to group the two positive examples together while also grouping the two negative examples together.

Therefore, no single linear decision boundary can correctly classify all four XOR data points, making the XOR problem unsolvable by a single-layer perceptron. This limitation motivates the use of Multi-Layer Perceptrons (MLPs), which can combine multiple decision boundaries to form a nonlinear boundary capable of correctly separating the XOR classes.

8. Discuss how introducing a hidden layer enables learning nonlinear decision boundaries.

A Multi-Layer Perceptron (MLP) overcomes the limitations of a single-layer perceptron by introducing one or more hidden layers between the input and output layers. These hidden layers enable the network to learn nonlinear decision boundaries, allowing it to solve problems such as XOR.

Each hidden neuron computes a weighted sum of the inputs, adds a bias, and applies a nonlinear activation function, such as the sigmoid function:

$$h_j = \sigma\left(\sum_{i=1}^{n} w_{ij}x_i + b_j\right)$$

The outputs of the hidden neurons are then combined by the output neuron to produce the final prediction:

$$y = \sigma\left(\sum_{j=1}^{m} v_j h_j + c\right)$$

Unlike a single-layer perceptron, where only one linear boundary can be learned, each hidden neuron learns a different linear boundary. The output neuron combines these boundaries to form a nonlinear decision boundary capable of correctly separating the XOR classes.

The sigmoid activation function introduces nonlinearity into the network. Without a nonlinear activation function, multiple layers would behave like a single linear layer and still be unable to solve the XOR problem.

During training, the network uses backpropagation and Binary Cross-Entropy (BCE) loss to update its weights and biases. Over successive iterations, the hidden layer learns meaningful feature representations that enable the network to correctly classify all four XOR input combinations.

Therefore, the addition of a hidden layer transforms the perceptron into a model capable of learning complex nonlinear relationships, making the XOR problem solvable.

CODE:

xor_boundry.py > ...

```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------
# Define the XOR Dataset
# ------------------------------------
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=np.float32)

# ------------------------------------
# Build the MLP Model
# ------------------------------------
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(8, activation='sigmoid'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

```

```
30
31  # ------------------------------------
32  # Compile the Model
33  # ------------------------------------
34  mode (function) optimizer: Any
35      optimizer='adam',
36      loss='binary_crossentropy',
37      metrics=['accuracy']
38  )
39
40  # ------------------------------------
41  # Train the Model
42  # ------------------------------------
43  model.fit(X, y, epochs=5000, verbose=0)
44
45  # ------------------------------------
46  # Create a Grid for Visualization
47  # ------------------------------------
48  x_min, x_max = -0.5, 1.5
49  y_min, y_max = -0.5, 1.5
50
51  xx, yy = np.meshgrid(
52      np.linspace(x_min, x_max, 300),
53      np.linspace(y_min, y_max, 300)
54  )
55
56  grid = np.c_[xx.ravel(), yy.ravel()]
57
58  # Predict for every point in the grid
59  Z = model.predict(grid, verbose=0)
60  Z = Z.reshape(xx.shape)
61
```

```python
# ------------------------------------
# Plot Decision Boundary
# ------------------------------------
plt.figure(figsize=(7, 7))

# Background decision regions
plt.contourf(xx, yy, Z, levels=50, cmap="coolwarm", alpha=0.5)

# Decision boundary line
plt.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)

# Plot XOR data points
plt.scatter(
    [0, 1],
    [0, 1],
    color='red',
    marker='x',
    s=150,
    linewidths=3,
    label='Class 0'
)

plt.scatter(
    [0, 1],
    [1, 0],
    color='blue',
    marker='o',
    s=150,
    label='Class 1'
)

```

```
    # Point labels
    plt.text(0.03, 0.03, "(0,0)", fontsize=10)
    plt.text(0.03, 1.03, "(0,1)", fontsize=10)
    plt.text(1.03, 0.03, "(1,0)", fontsize=10)
    plt.text(1.03, 1.03, "(1,1)", fontsize=10)

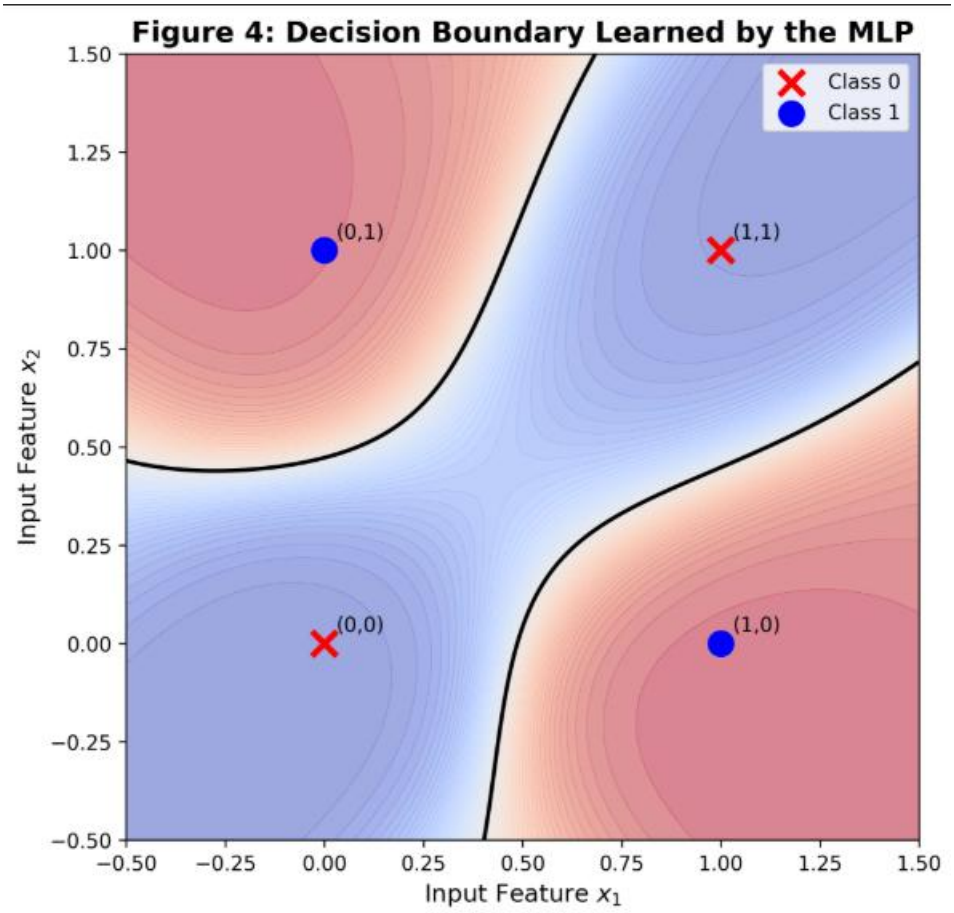
    # Labels and title
    plt.xlabel("Input Feature $x_1$", fontsize=12)
    plt.ylabel("Input Feature $x_2$", fontsize=12)
    plt.title(
        "Figure 4: Decision Boundary Learned by the MLP",
        fontsize=14,
        fontweight="bold"
    )

    # Legend
    plt.legend(loc="upper right")

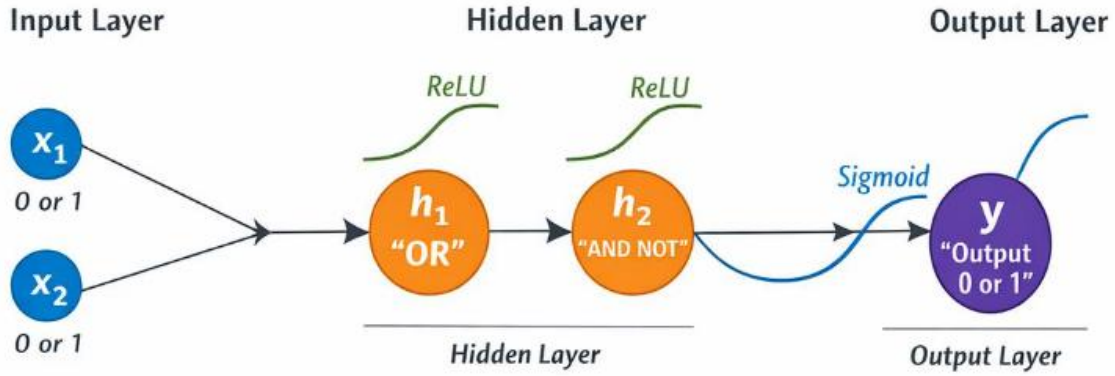
    # Save figure
    plt.savefig(
        "Figure_4_Decision_Boundary_Learned_by_MLP.png",
        dpi=300,
        bbox_inches="tight"
    )

    plt.show()
```

OUTPUT:


Figure 4: Decision Boundary Learned by the MLP
Class 0
Class 1
(0,1)
(1,1)
(0,0)
(1,0)
Input Feature x_2
Input Feature x_1
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50

9. Draw an MLP architecture capable of solving XOR.



10. Implement an XOR classifier using TensorFlow/PyTorch with one hidden layer, sigmoid activation, and Binary Cross Entropy loss.

CODE:

C: > Users > HARI RAMESH > Desktop > DL > DLLab > XOR > xor_implementation.py > ...

```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# ---------------------------------
# Set Random Seed
# ---------------------------------
tf.random.set_seed(42)
np.random.seed(42)

# ---------------------------------
# XOR Dataset
# ---------------------------------
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=np.float32)

# ---------------------------------
# Build the MLP Model
# ---------------------------------
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(8, activation='sigmoid'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# ---------------------------------
# Compile the Model
# ---------------------------------
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.05),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```python
# ---------------------------------
# Compile the Model
# ---------------------------------
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.05),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Display model summary
model.summary()

# ---------------------------------
# Train the Model
# ---------------------------------
history = model.fit(
    X,
    y,
    epochs=5000,
    verbose=0
)

# ---------------------------------
# Evaluate the Model
# ---------------------------------
loss, accuracy = model.evaluate(X, y, verbose=0)

print("\nModel Performance")
print("-" * 30)
print(f"Final Loss     : {loss:.6f}")
print(f"Final Accuracy : {accuracy * 100:.2f}%")

# ---------------------------------
# Predictions
# ---------------------------------
predictions = model.predict(X, verbose=0)

print("\nPredictions")
print("-" * 45)
print("{:<12}{:<12}{:<12}{:<10}".format(
    "Input", "Expected", "Probability", "Class"))

for inp, actual, pred in zip(X, y, predictions):
```

```
78
79  for inp, actual, pred in zip(X, y, predictions):
80      probability = pred.item()
81      predicted_class = int(probability >= 0.5)
82
83      print("{!s:<12}{:<12}{:<12.4f}{:<10}".format(
84          inp,
85          int(actual.item()),
86          probability,
87          predicted_class
88      ))
89
90  # ---------------------------------
91  # Plot Training Loss
92  # ---------------------------------
93  plt.figure(figsize=(7,5))
94  plt.plot(history.history['loss'], linewidth=2)
95
96  plt.title("Figure 2: Training Loss for XOR Classifier",
97            fontsize=14,
98            fontweight='bold')
99
100 plt.xlabel("Epoch")
101 plt.ylabel("Binary Cross Entropy Loss")
102
103 plt.grid(True, linestyle='--', alpha=0.5)
104
105 plt.tight_layout()
106
107 plt.savefig(
108     "Figure_2_Training_Loss_XOR.png",
109     dpi=300,
110     bbox_inches='tight'
111 )
112
113 plt.show()
```

OUTPUT:

```
Model: "sequential"
```

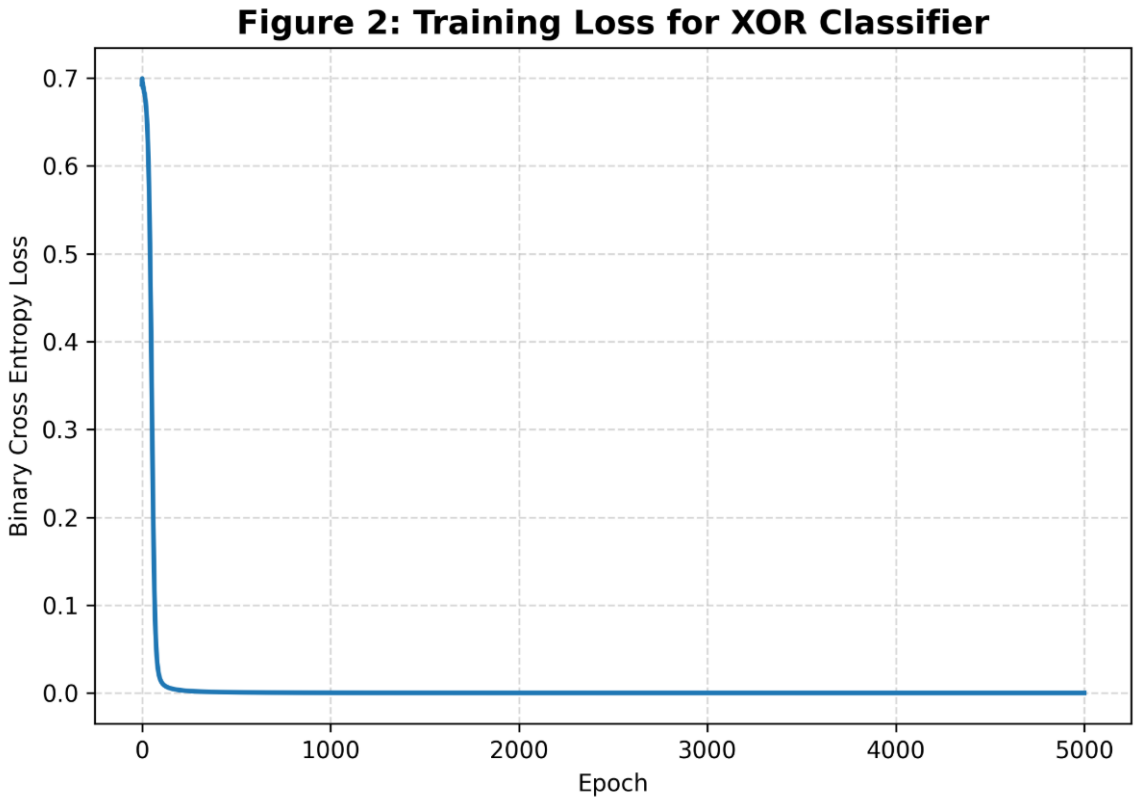
Layer (type)	Output Shape	Par… #
dense (Dense)	(None, 8)	24
dense_1 (Dense)	(None, 1)	9

```
 Total params: 33 (132.00 B)
 Trainable params: 33 (132.00 B)
 Non-trainable params: 0 (0.00 B)

Model Performance
------------------------------
Final Loss     : 0.000008
Final Accuracy : 100.00%

Predictions
--------------------------------------------
Input       Expected    Probability Class
[0. 0.]     0           0.0000      0
[0. 1.]     1           1.0000      1
[1. 0.]     1           1.0000      1
[1. 1.]     0           0.0000      0
```

11. Plot the training loss over epochs.



12. Report the final prediction accuracy.

```
Model: "sequential"
```

Layer (type)	Output Shape	Par… #
dense (Dense)	(None, 8)	24
dense_1 (Dense)	(None, 1)	9

```
Total params: 33 (132.00 B)
Trainable params: 33 (132.00 B)
Non-trainable params: 0 (0.00 B)

Model Performance
------------------------------
Final Loss     : 0.000008
Final Accuracy : 100.00%

Predictions
--------------------------------------------
Input       Expected    Probability Class
[0. 0.]     0           0.0000      0
[0. 1.]     1           1.0000      1
[1. 0.]     1           1.0000      1
[1. 1.]     0           0.0000      0
```

The trained Multi-Layer Perceptron achieved a **Binary Cross-Entropy loss of 0.000008** and a **prediction accuracy of 100%** on the XOR dataset. The network correctly classified all four possible XOR input combinations. The predicted probabilities are very close to the expected values of 0 and 1, indicating that the model has successfully learned the nonlinear XOR relationship.

Deliverables

- Mathematical proof
- Decision boundary visualization
- Code implementation
- Training curve
- Short discussion (200 words)

Exercise 2: Understanding Loss Functions (MSE vs BCE)

Objective

Compare Mean Squared Error (MSE) and Binary Cross Entropy (BCE) for binary classification.

Tasks

13. Given true labels [1,1,0,0] and predicted probabilities [0.95,0.80,0.30,0.10], compute the Mean Squared Error manually.

Given: True values (y) = [1, 1, 0, 0] Predicted values (y_pred) = [0.95, 0.80, 0.30, 0.10]

Formula: MSE = (1/n) * Σ(y - y_pred)^2

Step 1: Calculate squared errors

(1 - 0.95)^2 = 0.0025 (1 - 0.80)^2 = 0.0400 (0 - 0.30)^2 = 0.0900 (0 - 0.10)^2 = 0.0100

Step 2: Sum the squared errors

Sum = 0.0025 + 0.0400 + 0.0900 + 0.0100 Sum = 0.1425

Step 3: Divide by the number of observations

MSE = 0.1425 / 4 MSE = 0.035625

Final Answer: MSE = 0.035625

14. Compute Binary Cross Entropy manually.

Given:
True labels (y) = [1, 1, 0, 0]
Predicted probabilities (y_pred) = [0.95, 0.80, 0.30, 0.10]

Formula:
BCE = $-(1/n) * \Sigma[y * \ln(y_pred) + (1-y) * \ln(1-y_pred)]$

Here, n = 4.

Step 1: Calculate the loss for each observation.

For y = 1, y_pred = 0.95:
Loss = $-\ln(0.95) = 0.0513$

For y = 1, y_pred = 0.80:
Loss = $-\ln(0.80) = 0.2231$

For y = 0, y_pred = 0.30:
Loss = $-\ln(1 - 0.30) = -\ln(0.70) = 0.3567$

For y = 0, y_pred = 0.10:
Loss = $-\ln(1 - 0.10) = -\ln(0.90) = 0.1054$

Step 2: Sum the losses.

Sum = 0.0513 + 0.2231 + 0.3567 + 0.1054
Sum = 0.7365

Step 3: Calculate the average loss.

BCE = 0.7365 / 4
BCE = 0.1841

Final Answer: Binary Cross Entropy = 0.1841

15. Repeat the calculations for predictions [0.60,0.55,0.45,0.40].

Given:
True labels (y) = [1, 1, 0, 0]
Predicted probabilities (y_pred) = [0.60, 0.55, 0.45, 0.40]

PART A: Mean Squared Error (MSE)

Formula:
MSE = (1/n) * Σ(y - y_pred)^2

Step 1: Calculate squared errors.

(1 - 0.60)^2 = 0.16
(1 - 0.55)^2 = 0.2025
(0 - 0.45)^2 = 0.2025
(0 - 0.40)^2 = 0.16

Step 2: Sum the squared errors.

Sum = 0.16 + 0.2025 + 0.2025 + 0.16
Sum = 0.725

Step 3: Calculate MSE.

MSE = 0.725 / 4
MSE = 0.18125

Final Answer: MSE = 0.18125

PART B: Binary Cross Entropy (BCE)

Formula:
BCE = -(1/n) * Σ[y * ln(y_pred) + (1-y) * ln(1-y_pred)]

Step 1: Calculate the loss for each observation.

For y = 1, y_pred = 0.60:
Loss = -ln(0.60) = 0.5108

For y = 1, y_pred = 0.55:
Loss = -ln(0.55) = 0.5978

For y = 0, y_pred = 0.45:
Loss = -ln(1 - 0.45) = -ln(0.55) = 0.5978

For y = 0, y_pred = 0.40:
Loss = -ln(1 - 0.40) = -ln(0.60) = 0.5108

Step 2: Sum the losses.

Sum = 0.5108 + 0.5978 + 0.5978 + 0.5108
Sum = 2.2172

Step 3: Calculate BCE.

BCE = 2.2172 / 4
BCE = 0.5543

Final Answer: MSE = 0.18125
Binary Cross Entropy = 0.5543

16. Compare the values.

True Labels	Predicted Probabilities	Mean Squared Error (MSE)	Binary Cross Entropy (BCE)
[1,1,0,0]	[0.95, 0.80, 0.30, 0.10]	0.0356	0.1841
[1,1,0,0]	[0.60, 0.55, 0.45, 0.40]	0.1813	0.5543

Comparison:

The first set of predictions is closer to the true labels, resulting in lower values for both MSE and BCE. In the second set, the predicted probabilities are much closer to 0.5, indicating greater uncertainty. Consequently, both loss functions increase.

However, the increase in Binary Cross Entropy is much larger than the increase in Mean Squared Error. This is because BCE penalizes uncertain or incorrect predictions more heavily than MSE. As predictions move away from the correct class probabilities, BCE increases rapidly, whereas MSE grows more gradually.

Therefore, BCE is more sensitive to classification errors and provides a stronger learning signal during training.

17. Explain why BCE provides stronger gradients for classification.

Binary Cross Entropy (BCE) is specifically designed for binary classification tasks because it measures the difference between the predicted probability distribution and the true class labels. It produces larger loss values when the model makes confident but incorrect predictions.

For example, if the true label is 1 and the model predicts 0.01, the BCE loss becomes very large because

$$-\log(0.01) = 4.605$$

This large loss results in strong gradients, causing significant weight updates during backpropagation. As a result, the network learns to correct its mistakes more quickly.

In contrast, Mean Squared Error computes only the squared difference between the predicted probability and the true label. As predictions approach the extreme values of 0 or 1, the gradients produced by MSE become smaller, slowing down the learning process. This phenomenon is particularly noticeable when sigmoid activation functions are used, where MSE can suffer from vanishing gradients.

Because BCE generates stronger and more informative gradients, it enables faster convergence, more stable optimization, and generally better performance for binary classification problems. For this reason, Binary Cross Entropy is the preferred loss function for neural networks performing binary classification.

18. Train two identical neural networks: Model A (MSE) and Model B (BCE).

CODE:

train_model.py > ...

```python
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# ------------------------------------------------
# Generate Dataset
# ------------------------------------------------
X, y = make_moons(
    n_samples=1000,
    noise=0.25,
    random_state=42
)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```python
# ----------------------------------------------------
# Build Model
# ----------------------------------------------------
def build_model(loss_function):

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(2,)),
        tf.keras.layers.Dense(16, activation='relu'),
        tf.keras.layers.Dense(16, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss=loss_function,
        metrics=['accuracy']
    )

    return model

# ----------------------------------------------------
# Model A : MSE
# ----------------------------------------------------

model_mse = build_model("mse")

history_mse = model_mse.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=0
)

```

```python
# ----------------------------------------------------
# Model B : BCE
# ----------------------------------------------------

model_bce = build_model("binary_crossentropy")

history_bce = model_bce.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=0
)

# ----------------------------------------------------
# Evaluation
# ----------------------------------------------------

mse_loss, mse_acc = model_mse.evaluate(X_test, y_test, verbose=0)
bce_loss, bce_acc = model_bce.evaluate(X_test, y_test, verbose=0)

print("\nModel Performance\n")

print(f"MSE Accuracy : {mse_acc*100:.2f}%")
print(f"BCE Accuracy : {bce_acc*100:.2f}%")

```

```python
# ====================================================
# Plot 1 : Training Loss
# ====================================================

plt.figure(figsize=(8,5))

plt.plot(history_mse.history['loss'], label="MSE")
plt.plot(history_bce.history['loss'], label="Binary Cross Entropy")    Define
plt.title("Training Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.savefig("training_loss_comparison.png", dpi=300, bbox_inches="tight")

plt.show()


```

```python

# ================================================
# Plot 2 : Validation Accuracy
# ================================================

plt.figure(figsize=(8,5))

plt.plot(history_mse.history['val_accuracy'], label="MSE")
plt.plot(history_bce.history['val_accuracy'], label="Binary Cross Entropy")

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.savefig("validation_accuracy_comparison.png", dpi=300, bbox_inches="tight")

plt.show()


```

```python
# ================================================
# Plot 3 : Convergence Speed
# ================================================

plt.figure(figsize=(8,5))

plt.plot(history_mse.history['loss'], label="MSE")
plt.plot(history_bce.history['loss'], label="Binary Cross Entropy")

plt.xlim(0,30)

plt.title("Convergence Speed Comparison (First 30 Epochs)")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.savefig("convergence_speed_comparison.png", dpi=300, bbox_inches="tight")

plt.show()
```

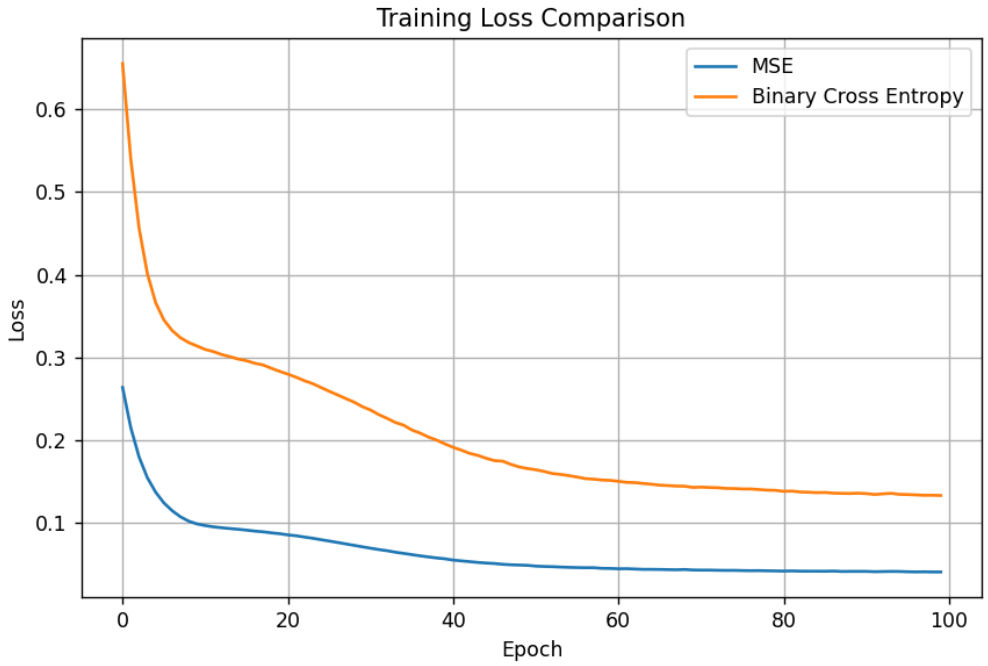
OUTPUT:

```
Model Performance

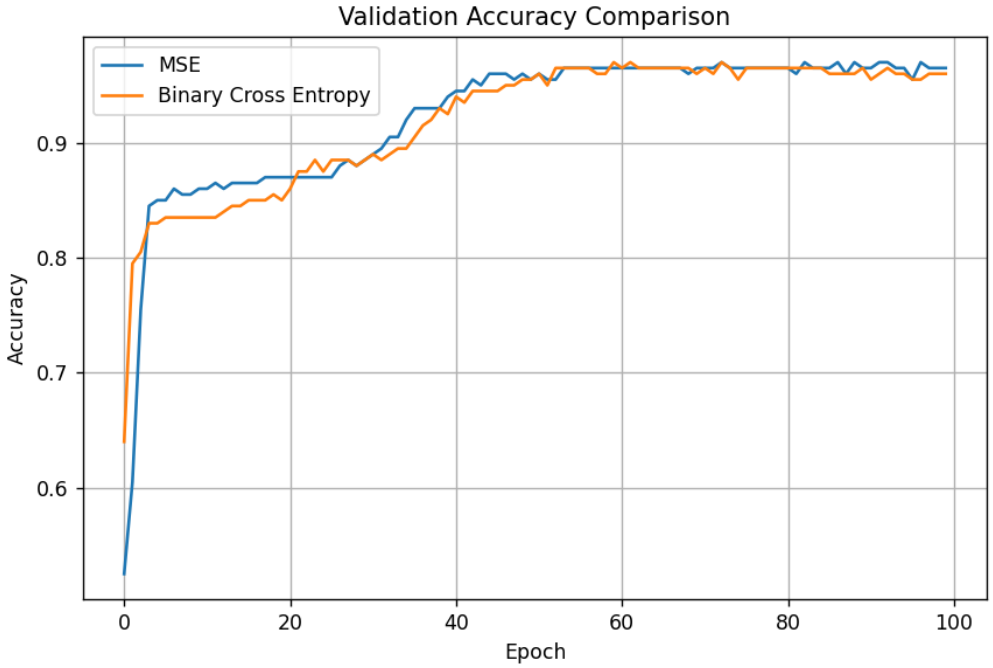
MSE Accuracy : 96.50%
BCE Accuracy : 96.00%
```

19. Compare training loss, accuracy, and convergence speed.

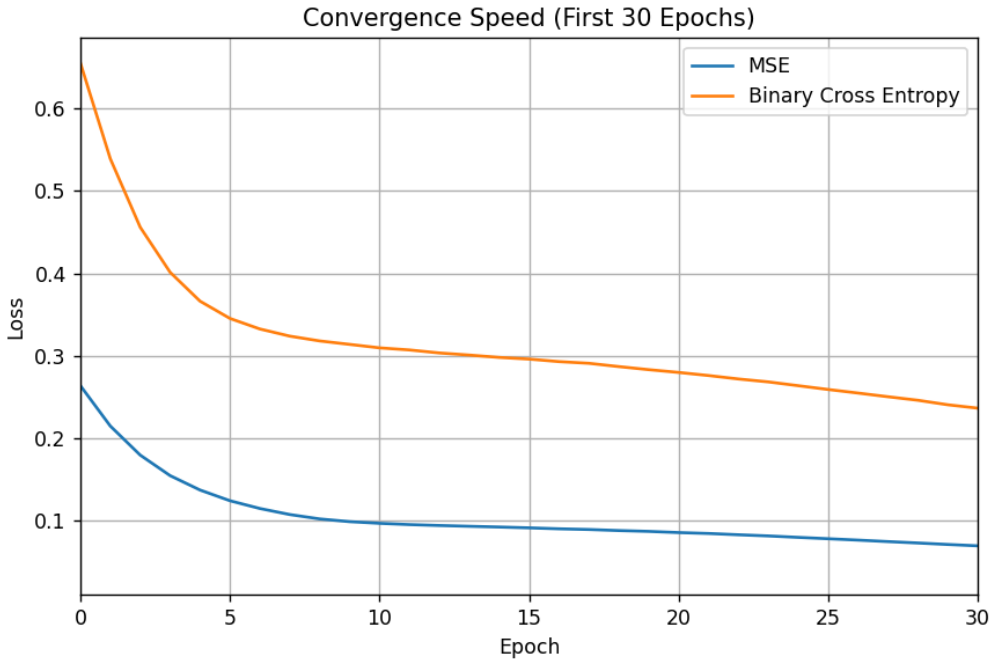
Training Loss:



Accuracy:



Convergence Speed:



Feature	MSE	BCE
Full Form	Mean Squared Error	Binary Cross Entropy
Main Use	Regression	Binary Classification
Output Layer	Linear output	Sigmoid output
Error Calculation	Squared difference between values	Difference between predicted probability and actual class
Gradient Strength	Weaker for classification	Stronger for classification
Convergence Speed	Slower	Faster
Accuracy Improvement	Gradual	Faster
Probability Prediction	Less suitable	More suitable
Training Efficiency	Lower for classification	Higher for classification
Best Application	Predicting continuous values	Classifying two classes

Deliverables

- Manual calculations
- Python implementation
- Comparison table
- Loss curve
- Conclusion

Exercise 3: Backpropagation from Scratch

Objective
Understand how gradients are propagated through a neural network.

Tasks

20. Use W1 = [[0.2,0.3], [0.4,0.1]], W2 = [[0.5], [0.6]], input x = [1,0], target y=1.

Perform forward propagation.

Compute hidden layer activations and output prediction.

Compute the loss.

Derive gradients manually using backpropagation.

Update the weights using gradient descent.

C) BACKPROPAGATION: CALCULATE GRADIENTS

Step 1: Calculate the gradient at the output neuron.

Loss function:
$L = (y - y_pred)^2$

Derivative of loss with respect to prediction:
$dL/dy_pred = 2(y_pred - y)$

$dL/dy_pred = 2(0.6501 - 1)$
$dL/dy_pred = -0.6998$

Step 2: Calculate the derivative of sigmoid at the output.

Sigmoid derivative:
$\sigma'(z) = \sigma(z)(1 - \sigma(z))$

$\sigma'(z_out) = y_pred(1 - y_pred)$

$\sigma'(z_out) = 0.6501 \times (1 - 0.6501)$
$\sigma'(z_out) = 0.2275$

Step 3: Calculate the output error term.

$\delta_out = (dL/dy_pred) \times \sigma'(z_out)$

$\delta_out = -0.6998 \times 0.2275$

$\delta_out = -0.1592$

Step 4: Calculate gradients for W2.

Gradient formula:
$dL/dw = \delta_{out} \times$ hidden activation

For weight w1 = 0.5:
$dL/dw1 = -0.1592 \times 0.5498$
$dL/dw1 = -0.0875$

For weight w2 = 0.6:
$dL/dw2 = -0.1592 \times 0.5744$
$dL/dw2 = -0.0914$

Therefore:
$dL/dW2 = [[-0.0875], [-0.0914]]$

Step 5: Calculate hidden layer error terms.

For hidden neuron H1:

$\delta_{h1} = \delta_{out} \times w1 \times h1(1 - h1)$

$\delta_{h1} = -0.1592 \times 0.5 \times 0.5498 \times (1 - 0.5498)$

$\delta_{h1} = -0.0197$

For hidden neuron H2:

$\delta_{h2} = \delta_{out} \times w2 \times h2(1 - h2)$

$\delta_{h2} = -0.1592 \times 0.6 \times 0.5744 \times (1 - 0.5744)$

$\delta_{h2} = -0.0233$

Step 6: Calculate gradients for W1.

Gradient formula:
$dL/dw =$ input $\times$ hidden error term

For weights connecting input X1 to hidden neurons:

$dL/dw11 = 1 \times (-0.0197)$
$dL/dw11 = -0.0197$

$dL/dw12 = 1 \times (-0.0233)$
$dL/dw12 = -0.0233$

For weights connecting input X2 to hidden neurons:

$dL/dw21 = 0 \times (-0.0197) = 0$

dL/dw22 = 0 × (-0.0233) = 0

Therefore:
dL/dW1 = [[-0.0197, -0.0233], [0, 0]]

D) UPDATE WEIGHTS USING GRADIENT DESCENT

Learning rate:
η = 0.1

Weight update formula:
New weight = Old weight - η × Gradient

Step 1: Update W1.

w11_new = 0.2 - 0.1 × (-0.0197)
w11_new = 0.20197

w12_new = 0.3 - 0.1 × (-0.0233)
w12_new = 0.30233

w21_new = 0.4 - 0.1 × 0
w21_new = 0.4

w22_new = 0.1 - 0.1 × 0
w22_new = 0.1

Updated W1:
W1 = [[0.20197, 0.30233], [0.4, 0.1]]

Step 2: Update W2.

w1_new = 0.5 - 0.1 × (-0.0875)
w1_new = 0.50875

w2_new = 0.6 - 0.1 × (-0.0914)
w2_new = 0.60914

Updated W2:
W2 = [[0.50875], [0.60914]]

E) FINAL ANSWERS

1. Hidden layer activations:
   h1 = 0.5498
   h2 = 0.5744

2. Output prediction:
   y_pred = 0.6501

3. MSE loss:
   Loss = 0.1224

4. Gradients:
   dL/dW1 = [[-0.0197, -0.0233], [0, 0]]
   dL/dW2 = [[-0.0875], [-0.0914]]

5. Updated weights:
   W1 = [[0.20197, 0.30233], [0.4, 0.1]]
   W2 = [[0.50875], [0.60914]]

21. Verify the results using Python.
    CODE:

backpropagation.py > ...

```python
import numpy as np

# Inputs
x = np.array([[1],[0]])
y = 1

# Initial weights
W1 = np.array([[0.2,0.3],
               [0.4,0.1]])

W2 = np.array([[0.5],
               [0.6]])

lr = 0.1


def sigmoid(x):
    return 1/(1+np.exp(-x))


# Forward propagation

z1 = W1.T @ x
h = sigmoid(z1)

z2 = W2.T @ h
y_hat = sigmoid(z2)


# Loss

loss = 0.5*(y-y_hat)**2

```

```python
# Backpropagation

dL_dy = y_hat-y

delta2 = dL_dy*y_hat*(1-y_hat)

dW2 = h*delta2


delta1 = (W2*delta2)*(h*(1-h))

dW1 = x@delta1.T


# Weight update

W1 = W1-lr*dW1
W2 = W2-lr*dW2


print("Prediction:", y_hat)
print("Loss:", loss)

print("\nUpdated W1:")
print(W1)

print("\nUpdated W2:")
print(W2)
```

OUTPUT:

```
Prediction: [[0.65012359]]
Loss: [[0.06120675]]

Updated W1:
[[0.20098492 0.3011673 ]
 [0.4        0.1       ]]

Updated W2:
[[0.50437579]
 [0.60457164]]
```

Deliverables

- Complete mathematical derivation
- Updated weights
- Python verification